

Reviewer Pack — Minimal Order of Pseudorandom Permutation Groups Bounds ACC^0 ...

Entry 48e21dab060f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 17:40:32 UTC

Minimal Order of Pseudorandom Permutation Groups Bounds ACC^0 Lower Bound

Entry ID: 48e21dab060f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 17:40:32 UTC

1. Conjecture

Field A (mathematical branch): Permutation Group Theory **Field B** (complexity object): Complexity Theory: ACC^0 Circuit Complexity

Statement:

{‘s1’: ‘For a given explicit function f in P , the minimal order of a pseudorandom permutation group G that can be used to construct an ACC^0 circuit for f is $\Omega(n^{2/2})$.’, ‘s2’: ‘That is, if there exists a permutation group G of order less than $n^{2/2}$ that can be used to construct such a circuit, then $P = \text{NP}$.’, ‘s3’: ‘For any instance with size $n \leq 40$, the conjecture is testable in <240 seconds using pure Python.’}

Rationale (proposer’s reasoning):

{‘s1’: ‘Permutation group theory has rich structures that could potentially lead to new techniques for proving lower bounds in complexity theory.’, ‘s2’: ‘Pseudorandom groups are a natural fit as they can be used to model randomness in computations, which is crucial for ACC^0 circuits.’, ‘s3’: ‘This conjecture explores the connection between group structures and circuit complexity, potentially uncovering new insights into ACC^0 lower bounds.’}

Taxonomy category: ACC_SIPSER (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 103779b46dee3b8e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if no permutation group G with order less than $n^{2/2}$ can be used to construct an ACC^0 circuit for any function f in P , measured across 30 seeds; it is falsified if any such group exists.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal order pseudorandom permutation groups" AND "ACC⁰ circuit complexity" - "pseudorandom permutation group" AND "complexity theory ACC⁰" - "construction ACC⁰ circuits" AND "permutation group theory"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_prime(n):
        if n <= 1:
            return False
        for i in range(2, int(math.sqrt(n)) + 1):
            if n % i == 0:
                return False
        return True

    def generate_primes(n):
        primes = []
        num = 2
        while len(primes) < n:
            if is_prime(num):
                primes.append(num)
            num += 1
        return primes

    def generate_permutation_group(order, n):
        # Generate a permutation group of the given order
        # This is a simplified version and may not be correct for all cases
        elements = list(range(n))
        random.shuffle(elements)
        return elements
```

```

def construct_acc0_circuit(group, n):
    # Construct an ACC0 circuit using the given permutation group
    # This is a simplified version and may not be correct for all cases
    if len(group) < n:
        return False
    return True

n = random.randint(5, 40)
primes = generate_primes(n)
min_order = float('inf')

for _ in range(30):
    order = random.randint(1, n**2 // 2 - 1)
    group = generate_permutation_group(order, n)
    if construct_acc0_circuit(group, n):
        min_order = min(min_order, order)

conjecture_holds = min_order >= n**2 / 2
counterexample = "" if conjecture_holds else f"Found a permutation group of order {min_order}"

return {
    "metric_name": "Minimum Order of Pseudorandom Permutation Group",
    "metric_value": min_order,
    "instances_tested": 30,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] if sys.argv[1:] else generate_primes(30)

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={results[0]['counterexample']}\\n first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
order 5 for n=21'}
TRIAL: {'metric_name': 'Minimum Order of Pseudorandom Permutation Group', 'metric_value': 4, 'instance': 1}
TRIAL: {'metric_name': 'Minimum Order of Pseudorandom Permutation Group', 'metric_value': 17, 'instance': 2}
TRIAL: {'metric_name': 'Minimum Order of Pseudorandom Permutation Group', 'metric_value': 1, 'instance': 3}
TRIAL: {'metric_name': 'Minimum Order of Pseudorandom Permutation Group', 'metric_value': 4, 'instance': 4}
TRIAL: {'metric_name': 'Minimum Order of Pseudorandom Permutation Group', 'metric_value': 25, 'instance': 5}
TRIAL: {'metric_name': 'Minimum Order of Pseudorandom Permutation Group', 'metric_value': 2, 'instance': 6}
TRIAL: {'metric_name': 'Minimum Order of Pseudorandom Permutation Group', 'metric_value': 102, 'instance': 7}
TRIAL: {'metric_name': 'Minimum Order of Pseudorandom Permutation Group', 'metric_value': 1, 'instance': 8}
TRIAL: {'metric_name': 'Minimum Order of Pseudorandom Permutation Group', 'metric_value': 19, 'instance': 9}
TRIAL: {'metric_name': 'Minimum Order of Pseudorandom Permutation Group', 'metric_value': 1, 'instance': 10}
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test has only used instances up to size $n=40$, which is too small to draw a definitive conclusion about the conjecture's validity. The metric does not scale trivially with n , but testing only up to $n=40$ may not be sufficient to capture all possible cases.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test has found a permutation group of order 1 for $n=33$, which is less than $n^{2/2}$ for $n=33$, thus falsifying the conjecture. | next: Increase the size of the instances tested to ensure that the conjecture holds for larger values of n . Additionally, consider testing with more seeds and different random number generators to confirm the counterexample.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14357
2	preregistration	ollama_remote	glm4:latest	0	0	5994
3	novelty	ollama_remote	glm4:latest	0	0	4660
4	novelty	ollama_remote	glm4:latest	0	0	5487
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14266
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7523
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9454
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8515
9	critic	ollama_remote	glm4:latest	0	0	10828
10	judge	ollama_remote	glm4:latest	0	0	5957

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 87040 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in <research/audit/48e21dab060f.jsonl>)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/48e21dab060f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/48e21dab060f.tar.gz` (if generated)